05 멀티플레이 구조 — Listen Server

한 플레이어가 호스트가 되어 서버와 클라이언트를 동시에 수행하고,
나머지 플레이어가 접속하는 구조를 채택했습니다.

Listen Server

채택

호스트가 서버+클라이언트 역할을 함께 수행

+ 별도 서버 불필요, 구현·배포 단순

– 호스트 의존 · 호스트 부하

Dedicated Server

전용 서버가 게임을 전담 호스팅

+ 공정·안정, 강한 치팅 방어

– 별도 인프라·운영 비용

P2P

클라이언트끼리 직접 통신

+ 서버 비용 없음

– 동기화·보안·NAT, Authority 부재

선택 이유 소규모 협동(3인) 프로젝트 범위에 적합하고, 별도 인프라 없이 빠르게 구현·테스트할 수 있어 Listen Server를 채택

06 RPC 프레임워크 — 누가 실행하는가

멀티플레이 동기화의 핵심 질문: **“이 코드는 누가 실행해야 하는가?”**

Server RPC

Client → Server

클라이언트가 서버에 “이 행동을 해도 될까요?” 요청. 보안과 직결됩니다.

적용 사례 **주포 발사 (Fire Shell)**

좌클릭해도 포탄을 직접 스폰하지 않고 Server_Fire RPC로 서버가 장탄수를 검증 후 스폰 → 무한 탄창 등 치팅을 원천 차단.

Owning Client

Server → 특정 Client

서버가 액터를 조종하는 특정 단일 클라이언트에게만 명령을 내립니다.

적용 사례 **HUD 데이터 갱신**

서버가 최종 연산한 체력·장탄수를 파라미터로 담아 Client_UpdateUI 호출 → 해당 플레이어에게만 반영, 지연으로 인한 UI 꼬임 방지.

07 RPC 프레임워크 — Multicast & 최적화

NetMulticast

Server → ALL (Server + Clients)

서버가 자신을 포함한 접속 중인 모든 플레이어에게 동시에 같은 명령을 실행시킵니다.

적용 사례 — 시청각 이펙트 동기화

- 포탄이 장갑에 적중했을 때 폭발 파티클(VFX) 표시
- 수리 중 타격음(SFX)을 주변 모든 플레이어에게 Multicast_PlayRepairSound로 동기화

Optimization Insight

모든 변수·이벤트를 리플리케이트하면 네트워크 대역폭이 초과됩니다. 물리와 데미지는 Server에서 통제하고, 순수 연출은 Multicast로 위임해 서버 부하를 최소화하는 정석적인 패턴을 따랐습니다.

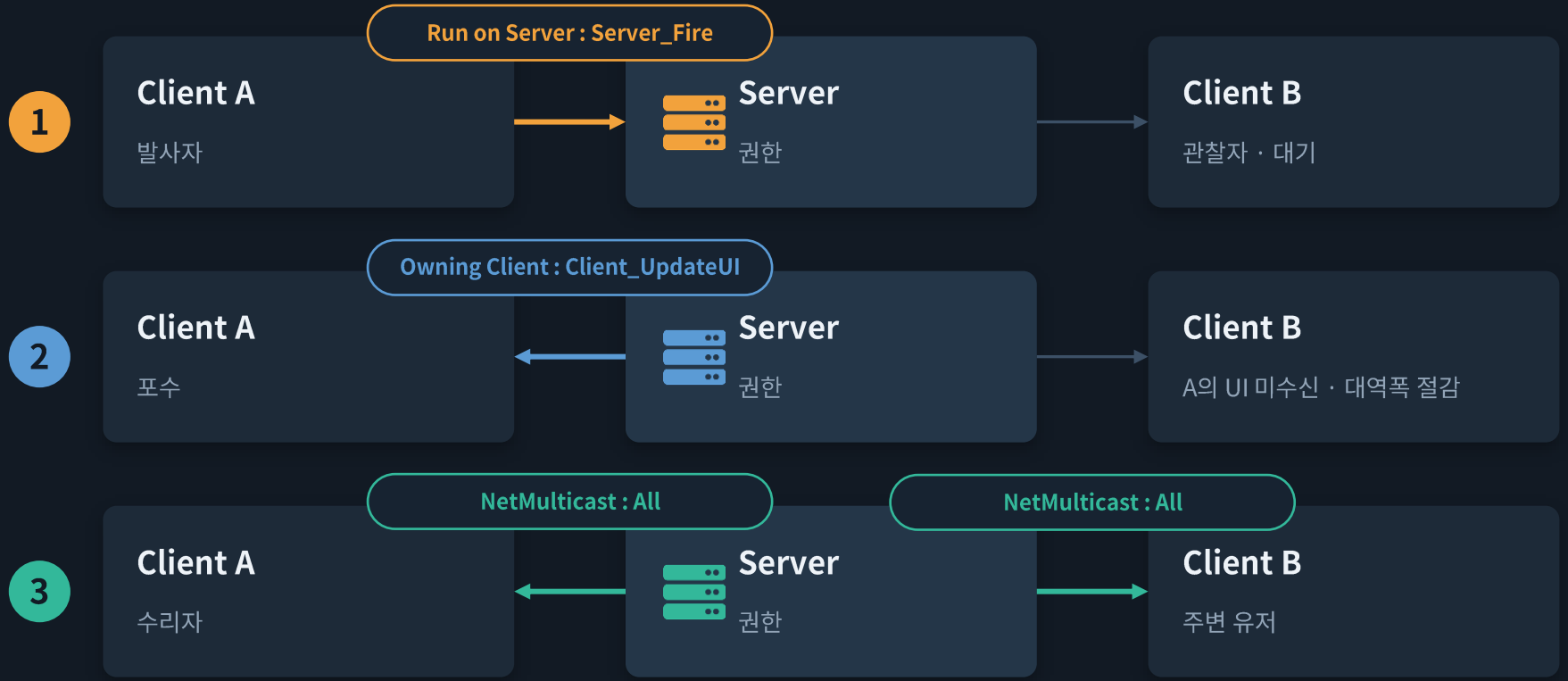
08 RPC 아키텍처 매트릭스

개발 중 발생한 동기화 누락·데이터 충돌 문제를 해결하기 위해 정립한 내부 통신 기준표

통신 방식 (RPC)	실행 주체	주요 사용 목적	Project Fortress 적용 예시
Run on Server	Server	클라이언트 행동을 서버가 검증·승인할 때	포탄 발사, 데미지 적용, 탑승/하차 요청
Owning Client	해당 액터의 Client	조종 중인 플레이어 본인에게만 필요한 피드백	포수 시점 전환, 나침반 UI 갱신
NetMulticast	Server + All Clients	월드 내 모두가 동일하게 봐야 하는 연출	전차 파괴 연출, 수리 타격음, 포탑 회전 동기화
RepNotify	변수 변경된 모든 Client	지연 참여(Late Join)에도 상태를 보장해야 할 때	파괴된 전차 잔해, 체력 바 동기화

09 RPC 이벤트 흐름

각 네트워크 노드 간의 이벤트 실행 흐름



10 Replication 으로 인한 버그 실제 사례

초기 구현에서 클라이언트–서버 동기화가 지켜지지 않았던 대표 사례입니다.

호스트만 상호작용 가능

문제 상호작용 로직이 서버(호스트)에서만 동작해, 일반 클라이언트는 좌석·오브젝트와 상호작용이 되지 않았습니다.

해결 상호작용 요청을 Server RPC로 위임하고 권한·오너십을 정리해 모든 클라이언트에서 동작하도록 수정.

대포 조작이 호스트에 미전달

문제 클라이언트에서 움직인 대포(포탑) 회전이 호스트로 전달되지 않아, 다른 플레이어 화면에 반영되지 않았습니다.

해결 클라이언트 입력 → Server RPC → Multicast 흐름으로 회전 값을 동기화해 모두에게 일관되게 반영.

11 AI 에셋 파이프라인 & 렌더링 최적화

Generative 3D 에셋 도입의 한계 극복과 퍼포먼스 확보 전략입니다.

1. Generative AI (Meshy)

빠른 프로토타이핑과 에셋 생산성을 위해 AI 3D 생성 툴로 전차·구조물 에셋을 생성.

문제 비정형 하이폴리곤 구조로 Draw Call·Triangle Count 폭발 위험.

2. Workflow Dilemma

기존 방식이면 수동 리토폴로지(Retopology)·LOD 제작 공수가 필수.

결정 인디 환경 한계상 AI의 본래 목적인 '생산성'을 위해 해당 공수를 과감히 생략.

3. UE5 Nanite

UE5의 가상화 마이크로폴리곤 지오메트리(Nanite)를 파이프라인에 전면 도입.

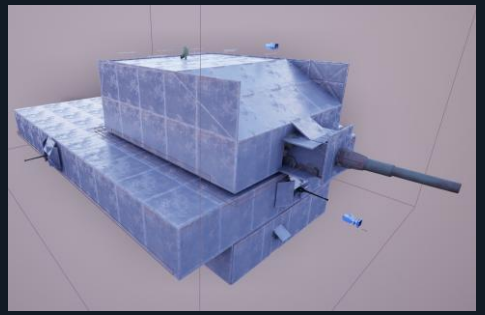
결과 수십만 폴리곤을 실시간 클러스터링해 디테일 손실 최소화하면서 부드러운 게임플레이 가능

Trade-off Management Nanite의 VRAM·스토리지 점유 증가를 통제하기 위해, 디테일이 중요한 전차·주요 배경 구조물에만 선별 적용해 메모리와 퍼포먼스의 균형을 확보.

12 이동 요새 구조 - 내/외부의 분리

“Proxy”역할의 탱크 메쉬

플레이어가 실제로 올라타 돌아다니는 요새(자세한 메쉬)와,
맵 위를 이동하는 요새를 분리해 구현.
내부 요새는 다양한 기구들과 플레이어들을 담고,
외부는 실제 이동/적과의 상호작용 담당





플레이어 탑승 공간

복잡한 메쉬 · 로컬 좌표에서 자유 이동

플레이어 행동 명령
(발사, 이동…)

피격/시야 정보,
주변 사운드

맵 상 이동 요새 (ExteriorPawn)

간단한 형상으로 맵을 실제 주행